Exploring Classical and Quantum Generative Adversarial Networks: An Experimental and Theoretical Study

Submitted by

Bhuvanesh O

Abstract

This project explores the development and training of a Quantum Generative Adversarial Network (QGAN), a hybrid model combining classical deep learning with quantum computing. By using a quantum generator and a classical discriminator, the QGAN attempts to learn and replicate simple data distributions. The model was trained on a scaled-down version of the MNIST dataset using limited qubits due to hardware constraints. Various challenges were encountered, such as unstable discriminator training, limited expressiveness due to few qubits, and prolonged quantum circuit evaluation time. Despite these limitations, the project provided valuable insights into the behavior of quantum models and their training dynamics, setting a foundation for further research in quantum-enhanced generative models.

Sno.	Title
1.	Introduction
2.	Literature Review / Theoretical Foundation
2.1	GANs (Generative Adversarial Networks)
2.2	Quantum Machine Learning
2.3	QGANs (Quantum GANs)
3.	Methodology
3.1	Tools & Frameworks
3.2	GAN Implementation
3.3	QGAN Implementation
3.4	Mathematical Foundation of GAN
4.	Experiments & Results
4.1	GAN Training
4.2	QGAN Training
4.3	Observations
5.	Challenges Faced
6.	Why Switch from GAN to QGAN? (Benefits of QGANs)
7.	Key Learnings
8.	Conclusion
9.	References

1. INTRODUCTION

Generative models are a powerful part of machine learning. Their main goal is to learn patterns from data and use that understanding to create new, realistic examples, like generating handwritten digits, images, or even text. One of the most well-known and effective types of generative models is the **Generative Adversarial Network (GAN)**.

A GAN works through a setup where it has two components — a **Generator**, which tries to create fake data that looks real, and a **Discriminator**, which tries to tell the difference between real and fake data. These two networks train together in a kind of minimax game, where the Generator keeps getting better at fooling the Discriminator. Over time, this process can result in realistic outputs. However, training GANs can be tricky. They often suffer from issues like instability, mode collapse (where the generator gets stuck producing limited variations), and sensitivity to training settings.

Along with the developments in classical machine learning, **Quantum Machine Learning (QML)** has started to gain attention. QML brings ideas from quantum computing, like superposition and entanglement into machine learning. This can offer new ways to process and represent information that aren't possible with traditional computing. One exciting idea from this area is the **Quantum GAN (QGAN)**, which blends quantum circuits into the GAN setup. These models aim to take advantage of quantum mechanics to possibly improve training or represent data more efficiently.

During this internship, I explored both classical GANs and QGANs. The goal was to understand how they work, implement both types of models, and compare their behavior on common datasets. The project included both self-guided learning and hands-on experimentation. It gave me the chance to look at how quantum concepts can be combined with deep learning and to see firsthand the strengths and limitations of each approach.

2. Literature Review / Theoretical Foundation

2.1 GANs (Generative Adversarial Networks)

GANs were introduced by Ian Goodfellow and his colleagues in 2014 and have since become one of the most popular generative modeling techniques in deep learning. A GAN consists of two neural networks: a **Generator (G)** and a **Discriminator (D)**. These two networks are trained in a **minimax game** framework.

The Generator tries to produce fake data (e.g., images) that resemble real data from a target distribution, while the Discriminator attempts to correctly classify inputs as either real or fake. During training, both networks are updated alternately: the Generator improves to fool the Discriminator, and the Discriminator learns to distinguish better. The goal is for the Generator to produce data so realistic that the Discriminator can no longer tell the difference.

Even though GANs were successful in generating sharp and realistic outputs, they are very difficult to train. Some of the common issues include:

- **Mode collapse** - where the Generator produces limited types of outputs.
- **Training instability** - due to the adversarial setup and non-convex optimization.
- **Sensitivity to hyperparameters and architecture choices** – Small changes in these settings can cause instability or degrade GAN performance.

As the years have passed, many different variations of GAN's have been found to solve these issues (ex: DCGAN, WGAN and StyleGAN)

2.2 Quantum Machine Learning

Quantum Machine Learning is a rapidly growing research area that explores how quantum computing can be used to enhance machine learning models. Quantum computers use **qubits** instead of classical bits, allowing them to represent and manipulate information using the principles of **superposition**, **entanglement**, and **quantum interference**.

These properties allow quantum algorithms to represent and process complex patterns more efficiently, at least in theory. Some promising directions in QML include:

- Quantum versions of supervised learning (quantum support vector machines).
- Variational Quantum Circuits (VQCs) for parameterized models.
- Hybrid quantum-classical models for tasks like classification and clustering.

At present most of the QML research is carried out on simulators or noisy intermediate-scale quantum (NISQ) devices.

What is a Qubit?

A qubit, or quantum bit, is the basic unit of quantum information — analogous to a classical bit, but with fundamentally different properties. While a classical bit can be in one of two states (0 or 1), a qubit can exist in a superposition of both 0 and 1 simultaneously:

A qubit → $\alpha|0\rangle + \beta|1\rangle$

Here:

- $|0\rangle$ and $|1\rangle$ are the basic quantum states,
- α and β are complex numbers called probability amplitudes, and
- The total probability must satisfy: $|\alpha|^2 + |\beta|^2 = 1$

Additionally, qubits can be **entangled**, allowing correlations between qubits that are not possible classically. These properties enable quantum circuits to explore high-dimensional spaces more efficiently than classical systems in certain tasks.

Superposition:

Superposition is a fundamental principle of quantum mechanics that allows a qubit to exist in **multiple states at the same time**. Unlike a classical bit, which can only be in one of two states 0 or 1 a qubit can be in a **linear combination** of both:

$$|\psi\rangle=\alpha|0\rangle+\beta|1\rangle$$

So, superposition allows quantum generators to explore multiple states simultaneously, enabling more expressive and compact representations of probability distributions especially useful for generative modeling.

This means that **until measured**, the qubit holds both possibilities and only **collapses** to one (0 or 1) when observed. For example, we can think of it as a coin spinning in the air. It's not strictly heads or tails, it's in a superposition of both until it lands.

Entanglement:

It is a phenomenon where two or more qubits become connected, so that the state of one instantly affects the state of another, no matter how far apart they are. Entangled qubits can represent complex relationships between data features. It helps in learning dependencies and structure that classical models might miss.

Interference:

In quantum circuits, qubit states are combined in ways that strengthen useful outcomes and cancel out others this is called interference. It helps to guide the quantum system toward the correct answer. This allows circuits to focus on good solutions over bad ones (like optimization).

In quantum circuits, we let all possible answers exist at once, and then use interference to cancel out the wrong answers and boost the right ones — like tuning a radio to get a clear signal.

2.3 QGANs (Quantum GANs)

Quantum GANs are a hybrid approach that aims to combine the generative capabilities of GANs with the expressive power of quantum systems. In a QGAN, either the Generator, the Discriminator, or both can be implemented using quantum circuits.

- In some designs, a **quantum Generator** produces quantum states, which are then measured and post-processed to generate classical data. (**as implemented in this project**)
- Alternatively, a **quantum Discriminator** may be used to distinguish between real and generated data using quantum operations.

QGANs can potentially represent more complex distributions with fewer parameters compared to classical GANs. However, training quantum components requires a different approach, often using **variational quantum circuits** optimized with classical methods (like gradient descent or parameter shift rules).

The main motivations behind QGANs include:

- Exploring quantum advantages in generative modeling.
- Understanding how quantum circuits can approximate complex data distributions.
- Studying the interplay between quantum entanglement and data representation.

While QGANs are still in the early research phase, they provide a valuable testbed for combining quantum and classical machine learning and pushing the limits of current hardware and algorithms.

3. Methodology

3.1 Tools & Frameworks

We used a combination of classical machine learning libraries and quantum computing frameworks to implement and experiment with both GAN and QGAN models. The key tools included:

- **Python** – The primary programming language for model development.
- **PyTorch** – Used to build and train classical GAN architectures due to its flexibility and GPU support.
- **PennyLane** – A quantum machine learning library that integrates with PyTorch, used for building hybrid quantum-classical models.
- **Qiskit** – Used to simulate quantum circuits when implementing quantum components.
- **Matplotlib** – For visualizing training loss, generated outputs, and performance comparisons.

3.2 GAN Implementation

The classical GAN was implemented using PyTorch, following the architecture proposed by Goodfellow et al. Both the Generator and Discriminator were built using fully connected neural networks, suitable for working with low-resolution image data.

- **Dataset Used:** MNIST dataset
- **Training Strategy:** The Generator was trained to produce synthetic data while the Discriminator was trained to differentiate real vs. fake data in a minimax way
- **Evaluation:** Performance was monitored using loss curves and visual inspection of generated outputs across epochs.

3.3 QGAN Implementation

The QGAN architecture integrated a quantum component, either in the Generator, Discriminator, or both. A variational quantum circuit was used to represent the quantum part, which was trained alongside a classical counterpart.

- **Quantum Circuit Design:** A parameterized quantum circuit (PQC) was used with rotation and entanglement gates. The number of qubits was kept small to maintain simulation efficiency.
- **Hybrid Setup:** Classical components (e.g., data encoding and loss calculation) were combined with quantum circuits using PennyLane's qnode interface.
- **Training:** The hybrid model was trained end-to-end using PyTorch optimizers. Gradients were calculated using the parameter-shift rule for quantum components.

3.4 Mathematical Foundation of QGAN

Quantum Generative Adversarial Networks (QGANs) extend classical GANs by incorporating quantum circuits into either the **generator**, **discriminator**, or both. While the high-level adversarial framework remains similar, the quantum-specific formulation introduces several key mathematical concepts and operations.

1. Adversarial Loss (GAN Foundation)

Like classical GANs, the QGAN optimizes the following minimax game between a generator G and a discriminator D:

Objective Function:

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))]$$

In a **hybrid QGAN**, the generator GGG is a **parameterized quantum circuit**, and the discriminator may be classical (e.g., a neural network in PyTorch).

2. Parameterized Quantum Circuits (PQC)

In QGANs, the generator is often built as a Parameterized Quantum Circuit (PQC):

- The PQC transforms an input state (like $|\mathbf{0}\rangle^{\otimes n}$) using unitary gates with trainable parameters θ.
- The quantum state is then measured to produce output data.

Mathematically, the output state is:

$$|\psi(\theta)\rangle = U(\theta)|0\rangle^{\otimes n}$$

Measurements from this state form the generator's output:

x = Measure(|ψ(θ)⟩)

This data is passed to the classical discriminator D(x), which tries to distinguish between real and generated samples.

3. Gradient Computation: The Parameter-Shift Rule

Quantum circuits are not directly differentiable using classical backpropagation. Instead, QGANs rely on the Parameter-Shift Rule to compute gradients for updating θ:

If f(θ) is the expectation value of a quantum measurement, then:

$$\frac{d}{d\theta} f(\theta) = \frac{f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})}{2}$$

This allows us to calculate gradients by evaluating the quantum circuit at **two shifted values** of the parameter no need for analytical derivatives or symbolic math.

These gradients are then passed to a classical optimizer (like Adam or SGD) to update θ in the generator.

4. Output & Sampling

Quantum generators often output binary samples (e.g., bitstrings like 0011, 1100, etc.), or probabilities derived from measurement counts. These must be mapped to a meaningful representation for instance, pixel values in a scaled image dataset.

In some setups, classical post-processing layers are added after quantum sampling to match the data domain (e.g., reshape into image-like tensors).

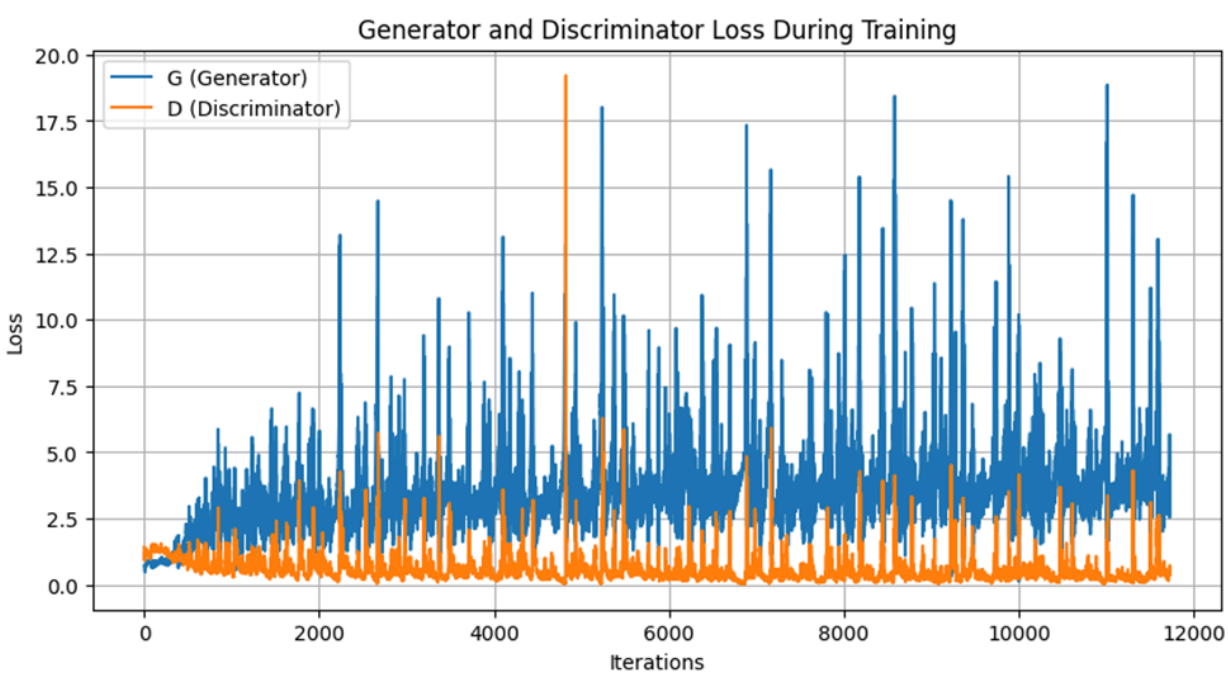
5. Full Hybrid Optimization Loop

1. **Sample z** from a prior (e.g., Gaussian or uniform).
2. **Encode z** into a quantum circuit (via angle encoding, etc.).
3. Run the PQC and **measure output G(z)**.
4. Pass G(z) to the **discriminator D**.
5. Compute the **loss**, using binary cross-entropy (BCE).
6. **Backpropagate loss to PQC parameters** using parameter-shift rule.
7. Update classical D and quantum G using optimizers.

4. Experiments & Results

4.1 GAN Training

The classical GAN was trained on the **MNIST** dataset for **25** epochs using a batch size of **128**. The training process involved alternating updates between the Generator and Discriminator using the Binary Cross Entropy Loss function and the Adam optimizer.



In this loss graph:

In this the **blue** line represents the **Generator Loss** where it shows a gradual increase in average loss with frequent spikes. This indicates that the generator is Struggling to fool the discriminator consistently. This could be due to **training instability**, a common challenge in GANs.

In this the **orange** line **Discriminator Loss** where it shows that it remains relatively low with occasional spikes. This suggests that the Discriminator is **winning** most of the time, it is easily able to distinguish between real and fake data.

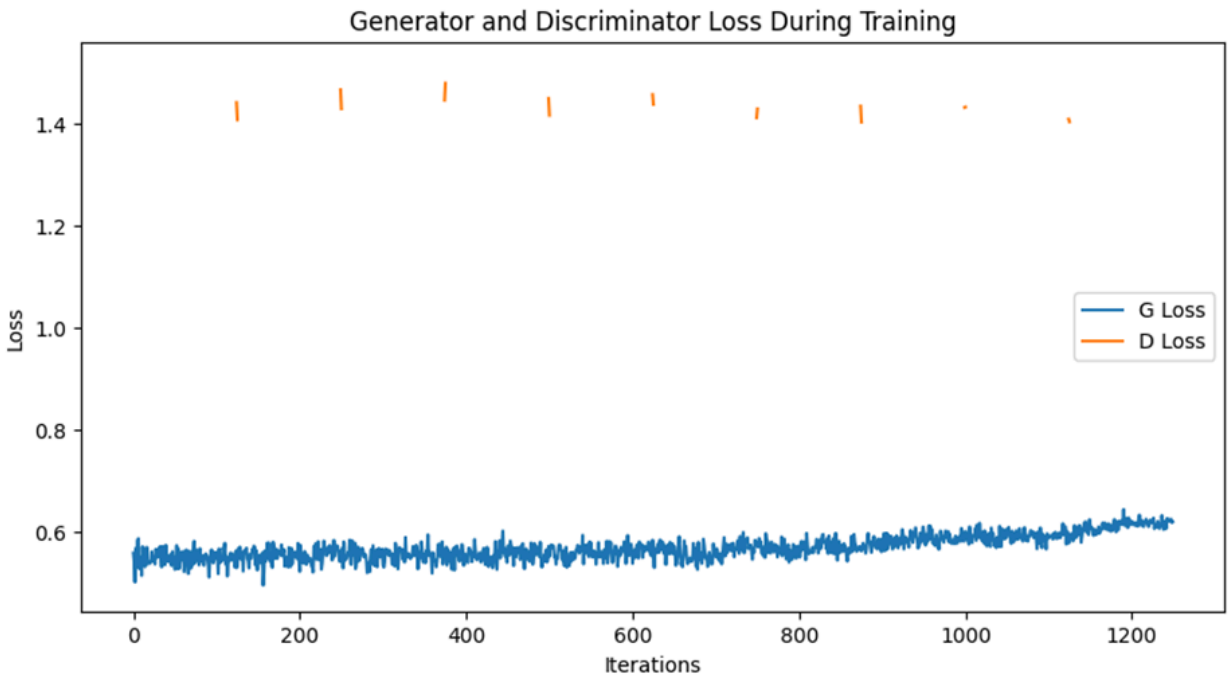
Frequent spikes and instability in the Generator loss suggest the need for further tuning of hyperparameters or the introduction of stabilization techniques such as label smoothing, alternative loss functions, or modified training schedules.

Although training for more epochs could potentially lead to better convergence, it was limited by computational constraints and the processing capacity of my system, which made extended training time-consuming.

4.2 QGAN Training

For QGAN training, a hybrid quantum-classical setup was implemented using **PennyLane & PyTorch**. The Generator was designed as a quantum circuit using parameterized quantum gates, while the Discriminator remained a classical neural network.

- **Quantum Generator:** Implemented using a variational quantum circuit with 4 & 8 (I have tried both) qubits and **number of layers as 2 & 8** (respectively) **layers** of parameterized gates (e.g., RX, RY, CNOT).
- **Training Challenges:** QGANs were more sensitive to learning rates and took longer to converge due to limitations in circuit depth and quantum simulator overhead.
- **Performance:** While convergence was slower and generated outputs were simpler compared to classical GANs, the experiment demonstrated that quantum circuits can represent meaningful generative patterns, even with shallow circuits.



In this QGAN plot we can see that **G Loss** is stable and slightly increasing. **D Loss** appears spiky and sparse, most likely only plotted at some update steps, which may imply that discriminator updates are infrequent or unstable.

The generator is improving slightly, whereas the Discriminator is unstable or updated too infrequently (possibly due to quantum circuit evaluation time). This could indicate **non-synchronized training steps** between G and D (which is quite common in QGANs if G involves quantum circuit sampling)

Quantum circuit evaluation time refers to the time it takes to execute a quantum circuit and measure its outputs, especially when training a quantum model (like a generator in a QGAN).

So, we can say that the QGAN training involved a quantum generator and classical discriminator. Due to circuit evaluation overhead, training was slower and limited to fewer epochs.

4.3 Observations

What we can say from these experiments is that:

- Classical GANs produced higher-quality and more detailed outputs, especially for complex image datasets.

- QGANs showed promise, but training was less stable and slower, mainly due to simulator limitations and noise.
- Hyperparameter tuning was significantly more sensitive in QGANs, and small changes could cause training to diverge.
- Both models confirmed the adversarial learning principle, with the Generator improving as the Discriminator became stronger.

5. Challenges Faced

1. Training Instability in GANs:

GANs are known for their adversarial nature, where two networks compete in a minimax game. This often leads to **training instability**—where the Discriminator overpowers the Generator or vice versa. Balancing the learning rates and loss convergence was difficult and required careful tuning of hyperparameters. Additionally, the model was prone to **mode collapse**, where the Generator starts producing repetitive outputs lacking diversity

2. Computational Limitations for Quantum Simulations

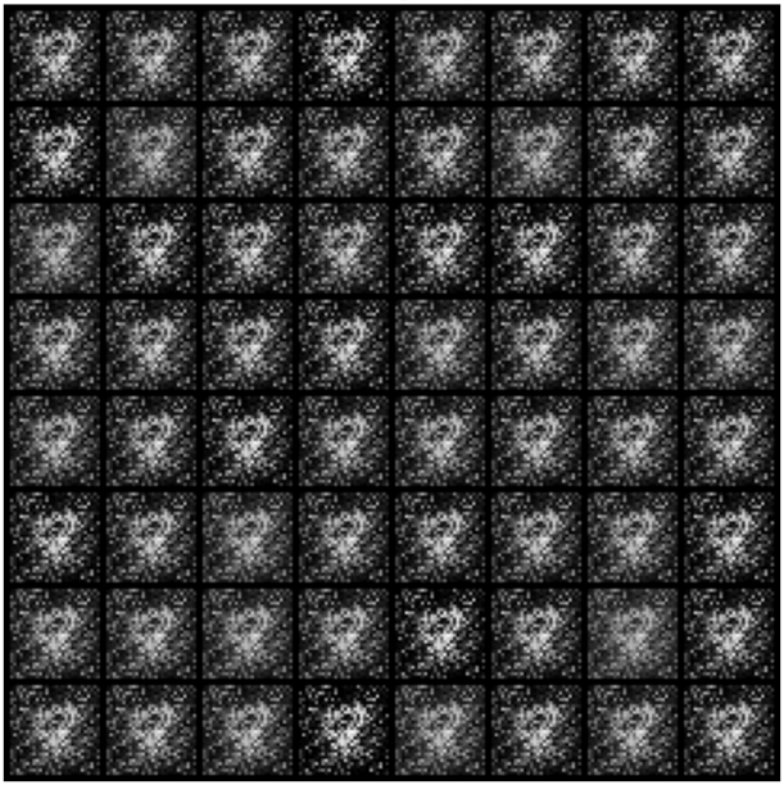
QGANs require quantum circuit evaluations, which even when simulated on classical machines are computationally expensive. Each forward pass involves simulating quantum gates and measurements, which **significantly increased training time**, especially when working with multiple qubits. Due to hardware limitations, it was not feasible to train for extended epochs or test larger architectures. As a result, the number of epochs was limited to 10 for both GAN and QGAN. Since I lack the hardware and I had no access to GPU, I used Google Collab's custom GPU where Google Colab provides access to Google computing resources.

3. Resource Constraints (Memory and Compute)

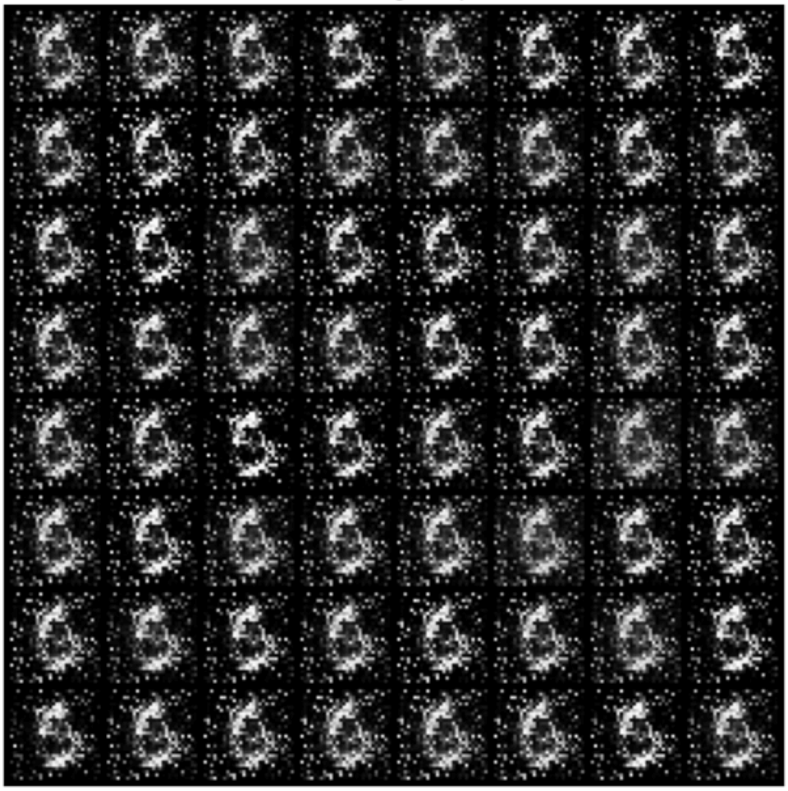
Training deep learning models on a local machine without GPU acceleration imposed further restrictions. **Batch sizes had to be kept small**, limiting the generalization capacity of the models. QGAN simulations were slow and memory intensive due to the exponential growth of state vectors with added qubits.

So, due to these constraints I had to limit my dataset for QGAN. Where I took a subset of the MNIST data of 1000 images (The MNIST data consists of about 60000 images) whereas I used the whole dataset for GAN. Which led to the following:

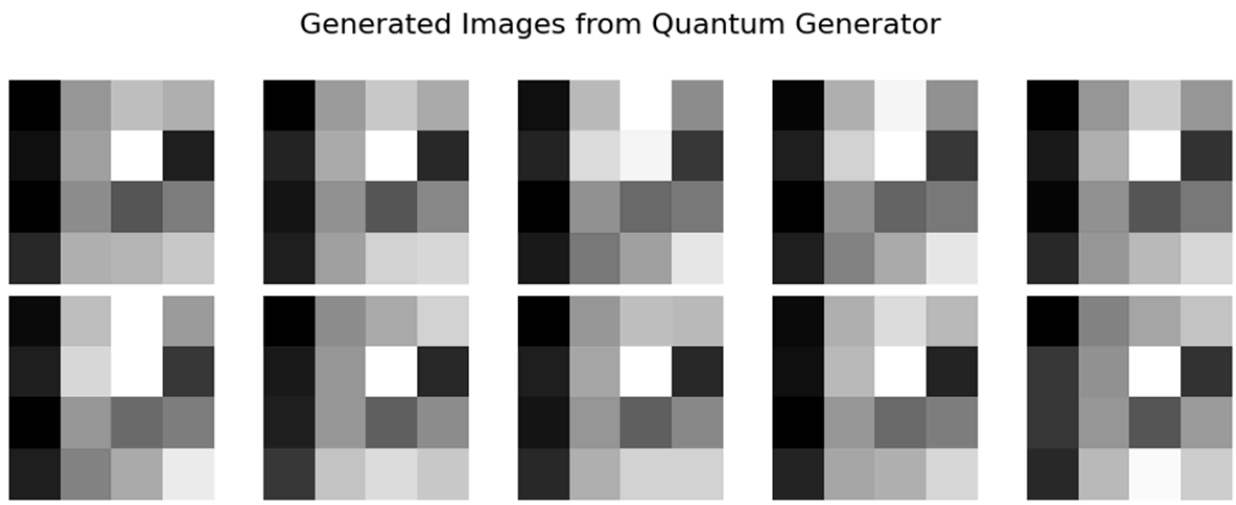
Generated Images (Epoch 1)



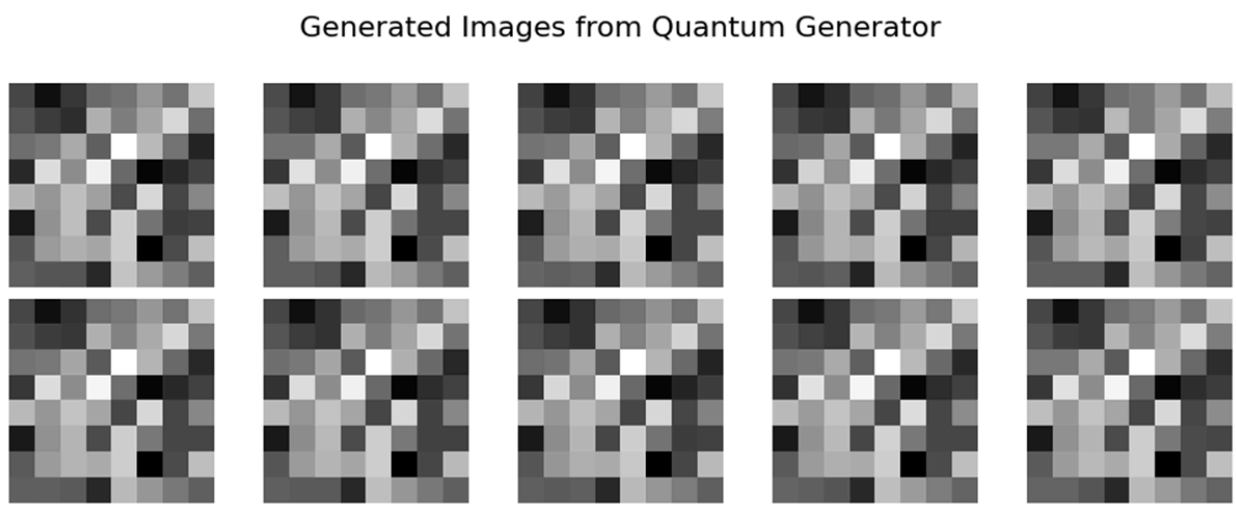
Generated Images (Epoch 22)



As shown in the figure these are the images generated by the classical GAN at epoch 1 (top image) and epoch 22 (bottom image) respectively. At epoch 1 we are not able to define what number it is representing. But then at epoch 22 we are able to notice that it closely resembles the digit 6



In the image shown above, the output generated by the Quantum GAN appears visually indistinct. This is primarily due to the use of only 4 qubits and a limited training subset (1,000 samples out of 60,000), which significantly constrains the model's expressive capacity. Additionally, image scaling may have further reduced visual clarity.



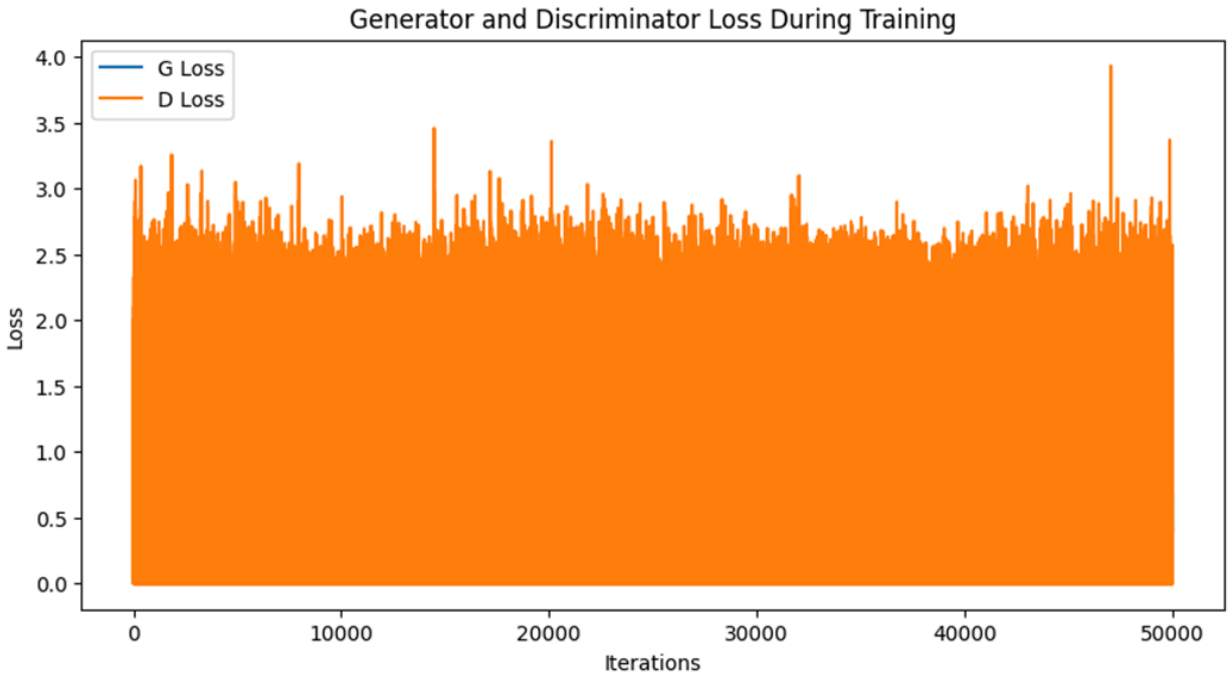
Similarly, the above image was generated by a Quantum GAN of 8 qubits with more layers for complexity

While the image generated using 8 qubits with additional layers shows slightly better structure compared to the 4-qubit output, it still lacks clear definition. This suggests that, although increasing the number of qubits and network depth enhances representational power, the current quantum circuit size and available quantum simulation resources are still insufficient for generating highly detailed outputs. Further improvements may be achieved by training on larger datasets, increasing the number of qubits, and leveraging more advanced quantum hardware.

Gradient Vanishing in Quantum Circuits

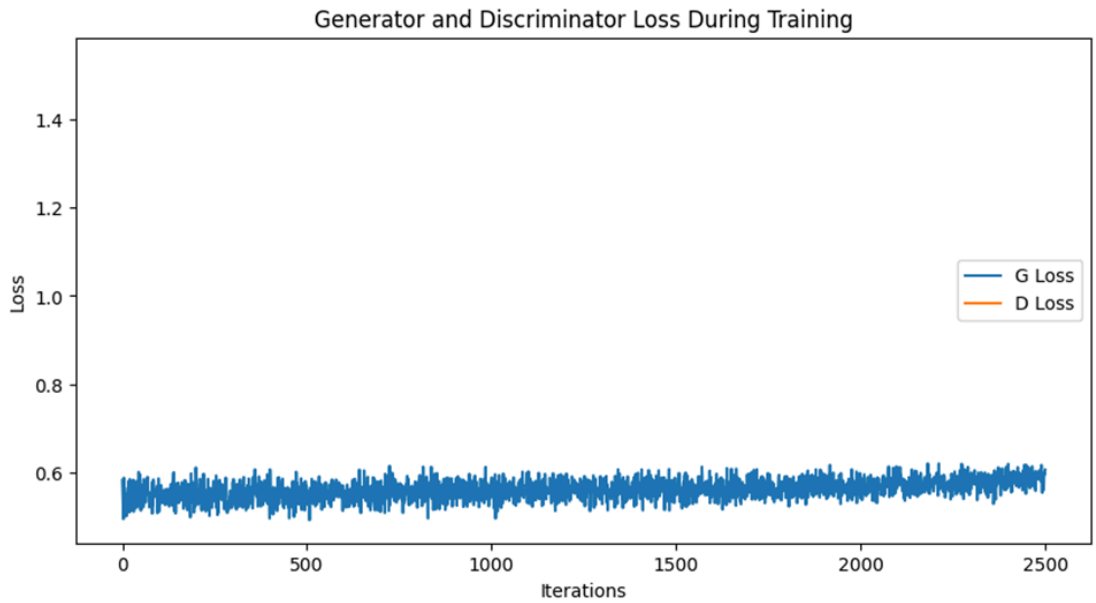
One of the major challenges encountered during QGAN training was the **vanishing gradient problem**, particularly in deeper quantum circuits. This phenomenon, often referred to as the **barren plateau problem**, causes the gradients to become extremely small, making it difficult for the model to update its parameters effectively during training.

For example, as the gradients were so small the model did not update any parameters and because of that the discriminator consistently classified generated images as fake, indicating lack of generator progress. This led to the following graph:

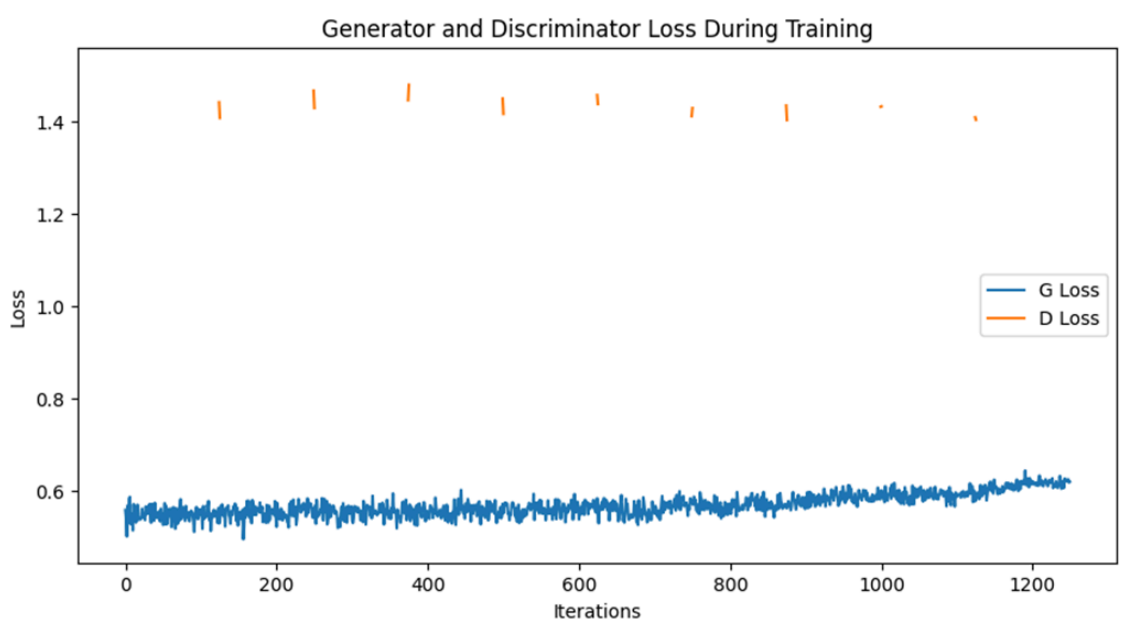


So, this indicates that **D is overpowering G**, making it too easy for D to distinguish real from fake. Where the **Generator may have collapsed**, i.e., stuck and not improving. Hence there could be vanishing gradients or learning imbalance. Which will happen mostly due to the improper gradient flow from the quantum generator.

To rectify this, I have changed the code so that there will be proper gradient flow which led to this graph :



In this graph we notice that the G loss and D loss are very close together. This suggests us that **a kind of equilibrium** has been reached, but it's somehow **too early and too flat**. Which likely means that **either both G and D are not learning well. Possibly,** the generator is just outputting static (similar or same) patterns, and the discriminator is unsure (unable to distinguish), This could be either due to Not enough quantum entanglement or noise in the system or maybe very low learning rate or poor optimization (both networks not improving). So, after varying the learning rates and improving the discriminator network it led to this graph:



So, we can say that in this QGAN plot, the Generator loss (G Loss) remains fairly stable and shows slight improvement over time. The Discriminator loss (D Loss), however, looks spiky and irregular — possibly because it's being updated less often or behaving unstably. This suggests that the Generator is learning slowly, but the Discriminator may not be improving well

But this could be improved by further increasing that size of the dataset (as I am using only 1000 MNIST images out of 60000) and by training more epochs

Plot	Generator Behavior	Discriminator Behavior	Likely Problem
1	Collapsed / not learning	Overfitting / dominating	Learning imbalance (D >> G)
2	Too stable	Too stable	Possibly both are stuck, poor loss signal
3	Slowly improving	Spiky, unstable	Asynchronous updates, unstable D

As a result, both the generator and discriminator struggled to improve, leading to stagnant loss curves and poor-quality outputs. This issue became more evident in runs with increased circuit depth or qubit count, where optimization became significantly harder.

6. Why Switch from GAN to QGAN? (Benefits of QGANs)

While classical GANs are powerful, they rely on large models and vast data to generalize well. Quantum GANs (QGANs) aim to leverage quantum phenomena such as superposition, entanglement, and quantum parallelism to improve generative learning, especially in low-data or high-dimensional scenarios.

Potential Advantages of QGANs:

- Exponential state space: With just **n** qubits, a quantum system can represent 2^n amplitudes, potentially encoding complex distributions more compactly
- Better generalization in small data regimes.
- **Hybrid learning flexibility**: Combining quantum generators with classical discriminators allows leveraging both paradigms.
- **Natural alignment with probabilistic modeling**: Quantum circuits inherently output probability distributions, which fits well with generative modeling tasks

7. Key Learnings

- **QGANs are highly sensitive to hyperparameters and architecture choices** even small changes in learning rate, optimizer, or circuit design can significantly affect training.
- **Training is slower compared to classical GANs**, primarily due to **quantum circuit evaluation time**, especially on simulators.
- **Gradient issues are common** the generator can get stuck producing static outputs, and the discriminator may become indecisive due to poor gradient flow.
- **Loss curves in QGANs are harder to interpret** because of quantum sampling noise and asynchronous updates between generator and discriminator.
- **Low number of qubits limits model expressiveness**, resulting in low-quality or indistinct generated images.
- **Quantum simulators slow down dramatically with more qubits** increasing from 4 to 8 qubits brings more complexity, but training time becomes a major bottleneck.
- **Discriminator updates may be sparse or unstable**, especially when quantum circuits are only partially optimized or take longer to evaluate.
- **Data reduction and scaling significantly impact outputs** working with a subset (e.g., 1,000 out of 60,000 images) limits diversity and generalization.
- **Quantum-classical integration is non-trivial** synchronizing training steps and gradient propagation across classical and quantum modules remains a technical challenge.
- **Visualization of results requires care** due to scaling artifacts and low resolution, it's often hard to visually evaluate generated outputs.
- **Quantum GANs are better suited for structured or simpler datasets** in their current state, due to hardware and software limitations.
- **Experimenting with deeper circuits or more complex layers often leads to instability**, requiring better optimization strategies or gradient clipping.
- **A balanced update strategy** (e.g., updating G and D alternately and equally) is crucial desynchronized updates lead to divergence.

8. Conclusion

In this project, we explored the training and implementation of Quantum Generative Adversarial Networks (QGANs) using a hybrid quantum-classical approach. Despite significant limitations such as low qubit count, limited dataset, and long training times due to quantum circuit evaluation, we were able to observe the fundamental learning dynamics of adversarial learning in a quantum setting. The results highlight the challenges of gradient flow, synchronization between generator and discriminator, and the need for careful architectural tuning.

While the generated images lacked clear visual structure, the project demonstrated the potential of QGANs and offered valuable insight into the intersection of quantum computing and generative modeling. With better hardware and optimization techniques, QGANs may evolve into powerful tools for high-dimensional data generation.

9. References

1. **Goodfellow, I. et al.** (2014). *Generative Adversarial Nets*. In Advances in Neural Information Processing Systems (NeurIPS). https://arxiv.org/abs/1406.2661

2. **Lloyd, S., Schuld, M., Ijaz, A., Izaac, J., & Killoran, N.** (2018). *Quantum Generative Adversarial Networks for Learning and Loading Classical Distributions*. https://arxiv.org/abs/1804.09139

3. **Dallaire-Demers, P., & Killoran, N.** (2018). *Quantum Generative Adversarial Networks*. Physical Review A, 98(1), 012324. https://arxiv.org/abs/1804.08641

4. **Benedetti, M., Garcia-Pintos, D., Nam, Y., Perdomo-Ortiz, A.** (2019). *A Generative Modeling Approach for Benchmarking and Training Shallow Quantum Circuits*. npj Quantum Information, 5(45). https://arxiv.org/abs/1801.07686

5. **PennyLane** Documentation. https://docs.pennylane.ai

6. **PyTorch** Documentation. https://pytorch.org/docs/stable/index.html

7. **Qiskit** Documentation. https://qiskit.org/documentation/